Organização do Repositório Git

Projeto	Formandos — gestão administrativa de formaturas e eventos
Marco do cronograma	22/09/2026 (Arquitetura inicial + organização do repositório Git)
Versão	1.0 — elaborada em 26/09/2026
Equipe	Luanderson Arlindo, Luiz Orlando, José Renato e Vinícius
Repositório	https://github.com/luandersonarlindo/Formandos

Este documento registra a organização do repositório do Formandos em 26/09/2026: a estrutura de pastas, a estratégia de ramos, a convenção de commits, o `.gitignore` e os scripts. O projeto **não tem hospedagem em produção** neste ciclo — marcado com † — e por isso o `.gitignore` guarda a linha `.vercel`, resíduo de uma plataforma de deploy que chegou a ser cogitada antes da decisão de ficar só no GitHub. O histórico de como essa decisão chegou até aqui está em **Kick-off**.

0. Visão geral

Item	Situação
Hospedagem	GitHub, `luandersonarlindo/Formandos` (`origin`)
Ramo principal	`main` (único ramo hoje)
Início do histórico	25/09/2026, com o commit criado pelo `create-next-app`
Commits até 26/09/2026	32, no padrão *Conventional Commits*
Licença de uso	Projeto acadêmico

1. Estrutura de pastas

```
Formandos/
├── README.md            visão geral, tecnologias, rotas, como executar
├── AGENTS.md            regras para quem programa (ler a documentação do Next.js 16)
├── package.json         dependências e scripts (dev, build, typecheck, test, db:*)
├── db/                  schema.sql, apply-schema.mjs e seed.mjs (banco de dados)
├── docs/                guia de estudo, catálogo de enquetes e entregas do
cronograma
│   └── entregas/        um subdiretório por data do cronograma, com .md e .pdf
├── public/              arquivos estáticos
├── scripts/
│   ├── teste/           dados de teste, capturas, auditoria e testes de fluxo
│   └── docs/            geração de PDF a partir dos documentos em Markdown
└── src/
    ├── app/             rotas: (publico), (onboarding), (app), (admin), (master),
api
    ├── actions/         Server Actions (mutações), uma por área
    ├── components/      ui/ (genéricos), features/ (de domínio) e vitrine/ (página
inicial)
    ├── lib/             consultas, regras, autenticação, e-mail e testes (*.test.ts)
    └── proxy.ts         checagem rápida de sessão
```

2. Estratégia de ramos

Hoje: todo o trabalho é feito em `main`.

Recomendado a partir da Sprint 2, com quatro pessoas:

Ramo	Uso	Exemplo
`main`	Sempre funcionando; é o que se apresenta	
`feat/<assunto>`	Uma funcionalidade	`feat/tema-escuro`
`fix/<assunto>`	Uma correção	`fix/id-repetido-duvida`
`docs/<assunto>`	Documentação	`docs/status-report`

Fluxo: criar o ramo a partir de `main` → commits pequenos → abrir um **Pull Request** → outra pessoa revisa → *merge* em `main` → apagar o ramo. Assim as quatro pessoas contribuem em paralelo, o histórico mostra quem fez o quê e a `main` não quebra.

3. Convenção de commits

Formato: `tipo(escopo): descrição no imperativo, em português`. O corpo do commit explica **o porquê**.

Tipo	Quando usar	Exemplo do histórico
`feat`	Funcionalidade nova	`feat(votacoes): votação em enquetes com voto identificado e alterável`

Tipo	Quando usar	Exemplo do histórico
`fix`	Correção de erro	`fix(duvidas): troca o id do campo de dúvida, que repetia o do conteúdo principal`
`docs`	Documentação	`docs: registra o login por email e senha e o envio de email`
`test`	Testes	`test: adiciona vitest e testes das funções puras`
`chore`	Manutenção, configuração, scripts	`chore(test): adiciona testes de fluxo no navegador`

Distribuição do histórico até 26/09/2026: 19 `feat`, 5 `docs`, 5 `chore`, 1 `fix`, 1 `test` e o commit inicial do gerador.

Regras:

1. Um commit = uma ideia. Não misturar correção e funcionalidade nova.
2. Nunca commitar sem rodar `npm run typecheck` e `npm test`.
3. Nunca commitar segredos (`.env.local`, chaves, `client_secret_*.json`).
4. Mensagem clara: quem lê o histórico entende o que mudou e por quê.

4. O que o Git ignora (`.gitignore`)

Padrão	Motivo
`/node_modules`, `/.next`, `/out`, `*.tsbuildinfo`, `next-env.d.ts`	Gerados na instalação ou no build
`.env*` (exceto `.env.example`)	Segredos locais
`client_secret_*.json`	Credenciais do Google
`scripts/teste/.tmp/`	Cookies e capturas dos testes
`.vercel` †	Sobras de hospedagem

O arquivo `.env.example` é o **modelo** das variáveis de ambiente; quem clona copia para `.env.local` e preenche.

5. Como começar (clonar e rodar)

```
git clone https://github.com/luandersonarlindo/Formandos.git
cd Formandos
npm install
cp .env.example .env.local        # preencher DATABASE_URL, BETTER_AUTH_SECRET etc.
npm run db:auth                   # cria as tabelas do Better Auth
npm run db:schema                 # cria as tabelas do domínio
npm run db:seed                   # carrega o catálogo padrão
npm run dev                       # http://localhost:3000
```

Antes de cada commit: `npm run typecheck` e `npm test`.

6. Scripts do `package.json`

Script	O que faz
`npm run dev`	Servidor de desenvolvimento
`npm run build` e `npm start`	Build e execução de produção
`npm run typecheck`	Gera os tipos das rotas e confere o TypeScript
`npm test`	Testes unitários (Vitest)
`npm run db:auth`, `db:schema`, `db:seed`	Banco de dados

7. Documentação no repositório

Documento	Onde
Visão geral, tecnologias, rotas e como executar	`README.md`
Guia de estudo do projeto	`docs/guia-de-estudo.md`
Catálogo de enquetes (fonte do *seed*)	`docs/catalogo-enquetes.md`
Entregas do cronograma (`.md` e `.pdf`)	`docs/entregas/`
Como testar as telas	`scripts/teste/README.md`

8. Situação e recomendações

- **Fato:** até 26/09/2026, todos os commits partem de uma única conta (`luandersonarlindo`), e há um único ramo.
- **Recomendação 1:** cada integrante escolher itens do *Sprint Backlog* e trabalhar em um ramo próprio, com Pull Request.
- **Recomendação 2:** proteger a `main` no GitHub, exigindo um Pull Request aprovado antes do *merge*.
- **Recomendação 3:** usar *Issues* e um quadro do GitHub Projects com as colunas do Sprint Backlog.
- **Recomendação 4:** marcar a entrega de cada sprint com uma *tag* (por exemplo, `sprint-1`).